

CAREER PREPARATION GUIDE

AI / ML Engineer Interview Prep

100 of the most frequently asked interview questions — with clear, concise answers — covering everything companies actually test for a Junior AI/ML Engineer role.

Python & Programming

Statistics

Machine Learning

Deep Learning

NLP

Computer Vision

Model Evaluation

MLOps

SQL

Behavioral

Table of Contents

01	Python & Programming Fundamentals	Q1–Q10
02	Statistics & Probability	Q11–Q20
03	Core Machine Learning	Q21–Q40
04	Deep Learning	Q41–Q55
05	Natural Language Processing	Q56–Q65
06	Computer Vision	Q66–Q70
07	Model Evaluation & Metrics	Q71–Q80
08	MLOps & Deployment	Q81–Q90
09	SQL & Data Handling	Q91–Q95
10	Behavioral & General Questions	Q96–Q100

How to use this guide

- Read one section per day rather than cramming everything at once — recall improves with spacing.
- For the coding-heavy sections, practice writing the algorithms out by hand, not just reading the answer.
- Adapt Section 10 (Behavioral) answers with your own real project details before the interview.

Python & Programming Fundamentals

Q1. What is the difference between a list, tuple, and set in Python?

A list is ordered and mutable, allowing duplicates. A tuple is ordered but immutable. A set is unordered, mutable, and stores only unique elements. Lists/tuples are used when order matters; sets are used for fast membership checks and deduplication.

Q2. What is the difference between deep copy and shallow copy?

A shallow copy creates a new object but inserts references to the same nested objects as the original. A deep copy recursively copies all nested objects, so changes to the copy don't affect the original at any level.

Q3. Explain Python's Global Interpreter Lock (GIL).

The GIL is a mutex that allows only one thread to execute Python bytecode at a time, even on multi-core systems. It simplifies memory management but limits true parallelism for CPU-bound tasks; multiprocessing or C-extensions (like NumPy) are used to work around it.

Q4. What are decorators in Python?

Decorators are functions that wrap another function to extend its behavior without modifying its code, commonly used for logging, timing, caching, or access control (e.g., `@staticmethod`, `@property`).

Q5. What is the difference between *args and **kwargs?

`*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dictionary. Both allow functions to accept a variable number of arguments.

Q6. How does Python manage memory?

Python uses automatic memory management via reference counting and a cyclic garbage collector to reclaim memory from objects with circular references.

Q7. What is a generator, and why use one?

A generator is a function using `yield` that produces values lazily, one at a time, instead of building a full list in memory. This is memory-efficient for large or infinite sequences.

Q8. Difference between == and is?

`==` checks value equality; `is` checks whether two references point to the same object in memory.

Q9. What are list comprehensions and why are they preferred?

List comprehensions provide a concise syntax to create lists (`[x**2 for x in range(10)]`). They are generally faster and more readable than equivalent for loops with `.append()`.

Q10. How would you handle missing values in a Pandas DataFrame?

Common approaches: `df.dropna()` to remove rows/columns, `df.fillna(value)` to impute with mean/median/mode or a constant, or using interpolation (`df.interpolate()`) for time-series data. The right approach depends on the amount and mechanism of missingness.

Statistics & Probability

Q11. What is the difference between population and sample?

A population includes all members of a defined group; a sample is a subset used to estimate population characteristics when studying the whole population is impractical.

Q12. Explain the Central Limit Theorem.

The CLT states that the sampling distribution of the sample mean approaches a normal distribution as sample size increases, regardless of the population's original distribution, provided samples are independent and identically distributed.

Q13. What is a p-value?

The p-value is the probability of observing results as extreme as (or more extreme than) the observed data, assuming the null hypothesis is true. A small p-value (typically < 0.05) suggests evidence against the null hypothesis.

Q14. What is the difference between Type I and Type II errors?

A Type I error is a false positive — rejecting a true null hypothesis. A Type II error is a false negative — failing to reject a false null hypothesis.

Q15. Explain correlation vs. causation.

Correlation measures the statistical association between two variables; causation implies one variable directly affects the other. Correlation does not imply causation because confounding variables or reverse causality may explain the relationship.

Q16. What is Bayes' Theorem?

Bayes' Theorem describes how to update the probability of a hypothesis given new evidence: $P(A|B) = [P(B|A) \times P(A)] / P(B)$. It's foundational to Naive Bayes classifiers and Bayesian inference.

Q17. What is the difference between variance and standard deviation?

Variance measures the average squared deviation from the mean; standard deviation is the square root of variance, expressed in the same units as the data, making it more interpretable.

Q18. What is a confidence interval?

A confidence interval gives a range of values, derived from sample data, that is likely to contain the true population parameter at a given confidence level (e.g., 95%).

Q19. Explain the difference between covariance and correlation.

Covariance measures the direction of the linear relationship between two variables but is unscaled. Correlation normalizes covariance to a range of -1 to 1, making it easier to interpret the strength and direction of the relationship.

Q20. What is skewness and kurtosis?

Skewness measures asymmetry of a distribution (left/right tail heaviness). Kurtosis measures the "tailedness" — how much of the data is in the tails versus the center, relative to a normal distribution.

Core Machine Learning

Q21. What is the difference between supervised, unsupervised, and reinforcement learning?

Supervised learning trains on labeled data to predict outcomes. Unsupervised learning finds patterns or structure in unlabeled data (e.g., clustering). Reinforcement learning trains an agent to take actions in an environment to maximize cumulative reward.

Q22. What is overfitting and how do you prevent it?

Overfitting occurs when a model learns noise/details specific to training data, hurting generalization. Prevention methods: regularization (L1/L2), cross-validation, dropout, early stopping, pruning, gathering more data, and simplifying the model.

Q23. What is the bias-variance tradeoff?

Bias is error from overly simplistic assumptions (underfitting); variance is error from sensitivity to small fluctuations in training data (overfitting). Total error = bias² + variance + irreducible error; the goal is to balance both.

Q24. Explain the difference between L1 and L2 regularization.

L1 (Lasso) adds the absolute value of coefficients as a penalty, encouraging sparsity (feature selection) by driving some weights to zero. L2 (Ridge) adds the squared value of coefficients, shrinking weights smoothly but rarely to exactly zero.

Q25. What is cross-validation?

Cross-validation splits data into multiple folds, training on some and validating on others across iterations (e.g., k-fold CV), giving a more robust estimate of model performance than a single train/test split.

Q26. How does a decision tree decide where to split?

It selects the feature/threshold that maximizes information gain (reduction in entropy) or minimizes Gini impurity, producing the purest possible child nodes.

Q27. What is Random Forest and how does it improve on a single decision tree?

Random Forest is an ensemble of decision trees trained on bootstrapped samples with random feature subsets at each split. Averaging (or majority voting) across trees reduces variance and overfitting compared to a single tree.

Q28. Explain Gradient Boosting.

Gradient Boosting builds trees sequentially, where each new tree corrects the residual errors of the previous ensemble by fitting to the negative gradient of the loss function. Examples: XGBoost, LightGBM, CatBoost.

Q29. What is the difference between bagging and boosting?

Bagging trains models in parallel on random subsets of data and averages results to reduce variance (e.g., Random Forest). Boosting trains models sequentially, where each model focuses on correcting previous errors, reducing bias (e.g., AdaBoost, XGBoost).

Q30. How does K-Nearest Neighbors (KNN) work?

KNN classifies a new point based on the majority class (or average value for regression) of its K nearest neighbors in feature space, using a distance metric like Euclidean distance. It's a lazy learner — no explicit training phase.

Q31. Explain how K-Means clustering works.

K-Means partitions data into K clusters by iteratively assigning points to the nearest centroid and updating centroids as the mean of assigned points, until convergence. It requires K to be chosen in advance (e.g., via the elbow method).

Q32. What is the curse of dimensionality?

As the number of features grows, data becomes sparse in high-dimensional space, distances become less meaningful, and models require exponentially more data to generalize well.

Q33. What is PCA (Principal Component Analysis)?

PCA is a dimensionality reduction technique that projects data onto orthogonal axes (principal components) that capture the maximum variance, reducing feature count while retaining most of the information.

Q34. What is feature scaling, and why is it important?

Feature scaling (e.g., normalization or standardization) transforms features to a similar range. It's important for distance-based algorithms (KNN, SVM, K-Means) and gradient-based optimization (neural nets, logistic regression) so no feature dominates due to scale alone.

Q35. What's the difference between normalization and standardization?

Normalization rescales data to a fixed range, typically [0,1]. Standardization transforms data to have a mean of 0 and standard deviation of 1, which is less sensitive to outliers than min-max scaling.

Q36. Explain how Support Vector Machines (SVM) work.

SVM finds the hyperplane that maximizes the margin between classes. For non-linearly separable data, it uses kernel functions (e.g., RBF, polynomial) to map data into higher dimensions where a linear separator can be found.

Q37. What is the difference between parametric and non-parametric models?

Parametric models assume a fixed functional form with a finite set of parameters (e.g., linear regression). Non-parametric models make fewer assumptions and can grow in complexity with data (e.g., KNN, decision trees).

Q38. What is class imbalance, and how do you handle it?

Class imbalance occurs when one class vastly outnumbers another. Solutions: resampling (oversampling minority via SMOTE, undersampling majority), class weighting, using appropriate metrics (F1, AUC-PR instead of accuracy), and ensemble methods.

Q39. What is regularization and why is it needed?

Regularization adds a penalty term to the loss function to discourage overly complex models, reducing overfitting and improving generalization to unseen data.

Q40. Explain the difference between a generative and a discriminative model.

Generative models learn the joint distribution $P(X, Y)$ and can generate new samples (e.g., Naive Bayes, GANs). Discriminative models learn the decision boundary $P(Y|X)$ directly, often achieving better classification performance (e.g., logistic regression, SVM).

Deep Learning

Q41. What is a neural network, in simple terms?

A neural network is a series of layers of interconnected nodes (neurons) that transform input data through weighted sums and non-linear activation functions to learn complex patterns and make predictions.

Q42. What is backpropagation?

Backpropagation computes the gradient of the loss function with respect to each weight by applying the chain rule backward through the network, enabling weight updates via gradient descent.

Q43. What is the vanishing/exploding gradient problem?

In deep networks, gradients can shrink (vanish) or grow (explode) exponentially as they backpropagate through many layers, making training unstable or slow. Solutions: proper weight initialization, batch normalization, residual connections, and using ReLU-family activations.

Q44. Explain the role of activation functions. Name a few.

Activation functions introduce non-linearity, allowing networks to learn complex patterns. Common ones: ReLU (fast, avoids vanishing gradients but can "die"), Sigmoid (bounded 0-1, used for binary output), Tanh (bounded -1 to 1), and Softmax (for multi-class probability output).

Q45. What is the difference between batch, mini-batch, and stochastic gradient descent?

Batch GD uses the entire dataset per update (accurate but slow). SGD updates using one sample at a time (fast but noisy). Mini-batch GD is a compromise, updating on small batches, balancing speed and stability — the most commonly used approach.

Q46. What is dropout, and why is it used?

Dropout randomly deactivates a fraction of neurons during training, forcing the network to not rely on specific neurons, which reduces overfitting and improves generalization.

Q47. What is batch normalization?

Batch normalization normalizes layer inputs (mean 0, variance 1) within each mini-batch, which stabilizes and speeds up training, allows higher learning rates, and acts as mild regularization.

Q48. Explain the difference between CNN and RNN.

CNNs (Convolutional Neural Networks) use convolutional filters to detect spatial patterns, ideal for images. RNNs (Recurrent Neural Networks) process sequential data by maintaining a hidden state across time steps, ideal for text/time-series.

Q49. What problem do LSTMs solve compared to vanilla RNNs?

LSTMs (Long Short-Term Memory networks) use gating mechanisms (input, forget, output gates) to control information flow, allowing them to retain long-term dependencies and avoid the vanishing gradient problem that plagues vanilla RNNs.

Q50. What is the Transformer architecture?

Transformers use self-attention mechanisms to weigh the relevance of all tokens in a sequence simultaneously (rather than sequentially like RNNs), enabling parallelization and better handling of long-range dependencies. It's the foundation of models like BERT and GPT.

Q51. Explain the attention mechanism.

Attention computes a weighted sum of value vectors, where weights (attention scores) are derived from the similarity between query and key vectors. This lets the model dynamically focus on the most relevant parts of the input for each output.

Q52. What is transfer learning?

Transfer learning reuses a model pretrained on a large dataset (e.g., ImageNet, or a large text corpus) and fine-tunes it on a smaller, task-specific dataset, saving training time and improving performance when labeled data is limited.

Q53. What is the difference between an epoch, batch, and iteration?

An epoch is one full pass through the entire training dataset. A batch is a subset of data processed together. An iteration is one update of model weights, corresponding to processing one batch.

Q54. What is the purpose of a loss function, and give examples.

A loss function quantifies the difference between predicted and actual values, guiding weight updates. Examples: Mean Squared Error (regression), Cross-Entropy Loss (classification), Hinge Loss (SVM).

Q55. What is weight initialization, and why does it matter?

Weight initialization sets the starting values of network weights before training. Poor initialization can cause vanishing/exploding gradients. Methods like Xavier/Glorot and He initialization scale weights based on layer size to keep gradients stable.

Natural Language Processing

Q56. What is tokenization?

Tokenization splits text into smaller units (words, subwords, or characters) that a model can process. Modern NLP models commonly use subword tokenization (e.g., Byte-Pair Encoding, WordPiece) to handle rare/unknown words.

Q57. What is TF-IDF?

TF-IDF (Term Frequency-Inverse Document Frequency) weights a word's importance in a document relative to a corpus — frequent words within a document get higher weight, but common words across all documents (like "the") get down-weighted.

Q58. What are word embeddings? Name examples.

Word embeddings are dense vector representations of words that capture semantic meaning, such that similar words have similar vectors. Examples: Word2Vec, GloVe, FastText, and contextual embeddings from BERT.

Q59. What is the difference between Word2Vec's CBOW and Skip-gram?

CBOW (Continuous Bag of Words) predicts a target word from surrounding context words. Skip-gram does the reverse — predicting context words from a target word. Skip-gram tends to work better on smaller datasets and rare words.

Q60. What is BERT and how is it different from GPT?

BERT (Bidirectional Encoder Representations from Transformers) is trained bidirectionally (looking at context from both directions) using masked language modeling, making it strong for understanding tasks. GPT is trained autoregressively (left-to-right) and is optimized for text generation.

Q61. What is Named Entity Recognition (NER)?

NER is the task of identifying and classifying entities in text (e.g., person names, organizations, locations, dates) into predefined categories.

Q62. What is perplexity in language models?

Perplexity measures how well a probability model predicts a sample; lower perplexity indicates the model is less "surprised" by the actual data, meaning better predictive performance.

Q63. What is the difference between stemming and lemmatization?

Stemming crudely chops word endings to get a root form (e.g., "running" → "run" using rules), which can produce non-words. Lemmatization uses vocabulary and grammar rules to return the proper dictionary base form (lemma), which is more accurate but slower.

Q64. What is RAG (Retrieval-Augmented Generation)?

RAG combines a retriever (which fetches relevant documents from a knowledge base) with a generator (an LLM) to produce answers grounded in retrieved context, reducing hallucination and enabling up-to-date, source-backed responses.

Q65. What is fine-tuning vs. prompt engineering for LLMs?

Fine-tuning updates model weights using additional task-specific data, changing the model's behavior directly. Prompt engineering crafts input instructions/examples to guide a frozen model's output without changing its weights — cheaper and faster but less deeply customized.

Computer Vision

Q66. What is a convolution operation in CNNs?

A convolution slides a small filter (kernel) across an image, computing dot products to detect local patterns like edges, textures, or shapes, producing a feature map.

Q67. What is pooling, and why is it used?

Pooling (e.g., max pooling, average pooling) downsamples feature maps, reducing spatial dimensions and computation while retaining the most important information, and adding translation invariance.

Q68. What is data augmentation, and why use it for images?

Data augmentation artificially expands the training set by applying transformations (rotation, flipping, cropping, color jitter) to existing images, improving model robustness and reducing overfitting, especially with limited data.

Q69. What is object detection vs. image classification vs. segmentation?

Classification assigns a single label to an entire image. Object detection identifies and localizes multiple objects with bounding boxes. Segmentation classifies every pixel, either grouping objects (semantic) or distinguishing individual instances (instance segmentation).

Q70. Name popular CNN architectures and their key contribution.

LeNet (early CNN for digit recognition), AlexNet (popularized deep CNNs with ReLU/dropout), VGG (deep, uniform 3x3 filters), ResNet (introduced residual/skip connections to train very deep networks), and EfficientNet (scaled depth/width/resolution efficiently).

Model Evaluation & Metrics

Q71. What is the difference between accuracy, precision, and recall?

Accuracy is the fraction of total correct predictions. Precision is the fraction of predicted positives that are actually positive ($TP / (TP+FP)$). Recall is the fraction of actual positives correctly identified ($TP / (TP+FN)$).

Q72. What is the F1 score, and when is it preferred over accuracy?

F1 score is the harmonic mean of precision and recall. It's preferred over accuracy on imbalanced datasets, where accuracy can be misleadingly high while the model fails to detect the minority class.

Q73. What is a confusion matrix?

A confusion matrix is a table showing counts of true positives, true negatives, false positives, and false negatives, giving a full picture of a classifier's performance beyond a single metric.

Q74. What is ROC-AUC?

The ROC curve plots True Positive Rate vs. False Positive Rate at various classification thresholds. AUC (Area Under the Curve) summarizes this into a single score from 0 to 1, where 1 is a perfect classifier and 0.5 is random guessing.

Q75. What are RMSE and MAE, and how do they differ?

RMSE (Root Mean Squared Error) squares errors before averaging, penalizing large errors more heavily. MAE (Mean Absolute Error) takes the average of absolute errors, treating all errors proportionally, making it more robust to outliers.

Q76. What is R-squared (R^2)?

R^2 represents the proportion of variance in the dependent variable explained by the independent variables in a regression model, ranging typically from 0 to 1 (higher is better fit).

Q77. What is the difference between validation set and test set?

The validation set is used during training to tune hyperparameters and make model selection decisions. The test set is held out entirely until the end, to give an unbiased estimate of final model performance.

Q78. What is data leakage, and how do you prevent it?

Data leakage occurs when information from outside the training set (often from the future or the test set) improperly influences the model, causing inflated performance estimates. Prevention: careful train/test splitting, doing preprocessing (scaling, imputation) only on training data, and avoiding target-derived features.

Q79. What is A/B testing, and how is it used in ML?

A/B testing compares two versions (A and B) of a model or feature by randomly assigning users/traffic to each and measuring performance on a business metric, to determine statistically which version performs better in production.

Q80. How do you evaluate a recommendation system?

Common metrics include Precision@K, Recall@K, Mean Average Precision (MAP), NDCG (Normalized Discounted Cumulative Gain), and online metrics like click-through rate (CTR) and conversion rate from A/B tests.

MLOps & Deployment

Q81. What is the ML lifecycle?

It typically includes: problem definition, data collection/cleaning, feature engineering, model training, evaluation, deployment, monitoring, and retraining — an iterative loop rather than a linear process.

Q82. What is model drift, and how do you detect it?

Model drift occurs when a model's performance degrades over time because the statistical properties of incoming data change (data drift) or the relationship between features and target changes (concept drift). It's detected by monitoring prediction distributions, input feature distributions, and live performance metrics against a baseline.

Q83. What is the difference between online and batch prediction?

Batch prediction generates predictions for a large dataset periodically (e.g., nightly), suited for non-time-sensitive use cases. Online (real-time) prediction serves individual requests immediately via an API, needed for latency-sensitive applications.

Q84. What is model versioning, and why does it matter?

Model versioning tracks different iterations of a trained model (along with its code, data, and hyperparameters) so teams can reproduce results, roll back to previous versions, and audit changes over time. Tools: MLflow, DVC.

Q85. What is containerization, and why is Docker used in ML deployment?

Containerization packages an application with all its dependencies into a portable, isolated unit. Docker ensures ML models run consistently across development, testing, and production environments, avoiding "it works on my machine" issues.

Q86. What is a feature store?

A feature store is a centralized repository for storing, sharing, and serving curated features for both training and real-time inference, ensuring consistency between training and serving pipelines and reducing duplicated feature engineering work.

Q87. What is CI/CD in the context of machine learning (MLOps)?

CI/CD (Continuous Integration/Continuous Deployment) automates the testing, validation, and deployment of ML code and models. In MLOps, this extends to also automatically validating data quality and retraining/redeploying models when performance degrades.

Q88. How would you scale a model to handle high inference traffic?

Approaches include model quantization/distillation to reduce size, batching requests, horizontal scaling with load balancers, using GPU/TPU-optimized serving frameworks (e.g., TensorRT, TorchServe, Triton), and caching frequent predictions.

Q89. What is A/B testing vs. shadow deployment for model rollout?

A/B testing splits live traffic between the old and new model and compares outcomes. Shadow deployment runs the new model alongside the current one in production without affecting real users, letting teams compare outputs safely before a full rollout.

Q90. What tools have you used (or would use) for experiment tracking?

Common tools: MLflow, Weights & Biases (W&B), Neptune.ai, and TensorBoard — they log hyperparameters, metrics, and artifacts across training runs for comparison and reproducibility.

SQL & Data Handling

Q91. What is the difference between INNER JOIN and LEFT JOIN?

INNER JOIN returns only rows with matching values in both tables. LEFT JOIN returns all rows from the left table, with NULLs for non-matching rows from the right table.

Q92. What is the difference between WHERE and HAVING?

WHERE filters rows before aggregation (GROUP BY); HAVING filters groups after aggregation, typically used to filter on aggregate functions like COUNT() or SUM().

Q93. What is a primary key vs. a foreign key?

A primary key uniquely identifies each row in a table. A foreign key is a column (or set of columns) in one table that references the primary key in another table, enforcing referential integrity between tables.

Q94. How would you find duplicate rows in a SQL table?

Typically by grouping on the relevant columns and filtering groups with a count greater than one:

```
SELECT column1, column2, COUNT(*)  
FROM table_name  
GROUP BY column1, column2  
HAVING COUNT(*) > 1;
```

Q95. What is indexing in databases, and why does it matter for ML pipelines?

An index is a data structure that speeds up data retrieval on a table at the cost of additional storage and slower writes. For ML pipelines, proper indexing on frequently filtered/joined columns significantly speeds up feature extraction from large datasets.

Behavioral & General Questions

Q96. Tell me about a machine learning project you worked on.

Structure your answer using the STAR method (Situation, Task, Action, Result): describe the problem, your specific role/approach, the models/techniques you used, and the measurable outcome or impact.

Q97. How do you stay updated with the latest developments in AI/ML?

Good answers mention specific habits: following research papers (arXiv, Papers with Code), following key conferences (NeurIPS, ICML, ACL), engaging with communities, taking courses, or building side projects with new tools/models.

Q98. Describe a time a model you built didn't perform as expected. What did you do?

Interviewers want to see debugging process and resilience: how you diagnosed the issue (data quality, leakage, wrong metric, insufficient data), what you changed, and what you learned from it.

Q99. How do you explain a complex ML model to a non-technical stakeholder?

Focus on communicating impact and intuition rather than math: use analogies, visualizations, and focus on what the model predicts, why it matters for the business, and its known limitations/confidence level — avoiding jargon.

Q100. Why do you want to work in AI/ML, and why this company specifically?

Tailor this to genuine motivation plus specific research into the company's products, ML use cases, or engineering culture — showing you've done homework and can connect your skills/interests to their actual work.

Final Tips Before Your Interview

- Practice explaining concepts simply — interviewers often care more about clarity of thought than jargon.
- Be ready to code core algorithms (e.g., KNN, gradient descent) from scratch in Python.
- Know your past projects in depth; be ready to discuss trade-offs and what you'd do differently.
- Ask clarifying questions on case-study/system-design style ML questions before diving into a solution.

Good luck with your interview!